

Universidad Internacional del Ecuador

Nombre: Gabriel Eduardo rios Fernández

Fecha: 15.5.2026

Enlace: <https://github.com/gariosfe/archivos-para-git.git>

Ficha de Impacto Arquitectónico

Descripción del caso

La empresa Spotify enfrentó problemas de escalabilidad y rendimiento debido al crecimiento masivo de usuarios en su plataforma de streaming musical. Inicialmente, utilizaba una arquitectura monolítica donde todos los servicios funcionaban en una sola aplicación centralizada.

Con el aumento de usuarios concurrentes, esta arquitectura comenzó a generar tiempos de respuesta elevados, caídas parciales del sistema y dificultades para desplegar nuevas funcionalidades. Para resolver el problema, Spotify migró progresivamente hacia una arquitectura basada en microservicios independientes desplegados en la nube.

Análisis de métricas de rendimiento y consumo afectadas

Métrica	Arquitectura Monolítica	Arquitectura de Microservicios
Tiempo de respuesta	Alto en horas pico	Reducido
Disponibilidad	Vulnerable a fallos globales	Alta tolerancia a fallos
Uso de CPU y memoria	Centralizado y saturado	Distribuido y optimizado
Escalabilidad	Vertical y costosa	Horizontal y flexible
Tiempo de despliegue	Lento	Rápido e independiente

La arquitectura monolítica obligaba a escalar toda la aplicación incluso cuando solo algunos módulos requerían más recursos, aumentando costos de infraestructura y consumo innecesario de CPU y memoria. Con microservicios, Spotify logró escalar únicamente los servicios más utilizados, optimizando recursos y mejorando el rendimiento general.

Justificación técnica

Desde el punto de vista arquitectónico, la migración permitió reducir cuellos de botella asociados al modelo de procesamiento centralizado descrito por el cuello de botella de von Neumann, donde múltiples procesos compiten por acceso a CPU y memoria.

La separación en microservicios mejoró la localidad de referencia porque cada servicio maneja conjuntos específicos de datos y procesos, reduciendo accesos innecesarios a memoria y aumentando la eficiencia de caché. Además, la jerarquía de memoria se aprovecha mejor al distribuir cargas de trabajo en diferentes nodos y servidores especializados.

Como trade-off, aunque la arquitectura distribuida incrementó la complejidad operativa y la comunicación entre servicios, Spotify obtuvo mayor disponibilidad, mejor escalabilidad y optimización del consumo de recursos computacionales.

Bibliografia

Fowler, M. (2014). *Microservices: A definition of this new architectural term*. Martin Fowler.
<https://martinfowler.com/articles/microservices.html>

Ford, N., Richards, M., Parsons, R., & Kua, P. (2021). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.